

Table of Contents

Especificación Técnica Completa

Sistema de Red de Referidos Multinivel con Reporte de Pagos Semanales

Documento de implementación dirigido a un modelo generador de código (Muse Spark 1.3). Este documento es la fuente única de verdad. Implementar TODO lo aquí descrito, sin omitir validaciones, índices, ni reglas de negocio. Donde se indique “configurable”, el valor NUNCA debe quedar escrito en el código (hardcodeado): debe leerse desde la tabla de configuración.

1. Resumen Ejecutivo

Se requiere una aplicación web (SaaS de un solo tenant) que modele una **red de referidos en forma de árbol**:

- Existe una **entidad raíz** (Persona #1, también llamada *ROOT* o *Administrador General*).
- La raíz invita hasta N **personas** (N por defecto = 12, **configurable** desde la interfaz).
- Cada persona invitada puede, a su vez, invitar hasta N personas.
- La profundidad es **ilimitada por diseño** (configurable: se puede imponer un límite máximo de niveles).
- Cada usuario entra **únicamente** mediante un **link de referido** (salvo la raíz, creada por seed).
- Cada usuario tiene un **portal personal** donde ve: sus datos, su link de referido, su árbol descendiente completo (no solo el nivel 1), la información de cada descendiente y los pagos reportados por su red.
- Cada usuario debe **reportar un pago/donación de forma semanal**.
- La raíz tiene un **dashboard financiero** con el detalle total: monto, fecha, autor, semana, estado de verificación, comprobante, y filtros por semana / rango / nivel / usuario / estado.

- **Todo parámetro de negocio es configurable por el usuario administrador** sin tocar código.
-

2. Stack Tecnológico (obligatorio)

Capa	Tecnología	Justificación
Framework	Next.js 15 (App Router) + TypeScript (strict)	SSR para dashboards, API Routes para backend, un solo despliegue
UI	React 19 + Tailwind CSS + shadcn/ui	Componentes accesibles, rápido de construir
Estado/datos cliente	TanStack Query v5	Caché, invalidación, paginación de tablas
Formularios	react-hook-form + Zod	Validación compartida cliente/servidor
Base de datos	PostgreSQL 16	Necesitamos CTE recursivas, ltree, transacciones serializables
ORM	Prisma 5 (+ SQL crudo para consultas recursivas)	Migraciones versionadas + escape a SQL cuando hace falta
Autenticación	Auth.js (NextAuth v5) con Credentials (email + password, Argon2id) + sesión JWT httpOnly	Control total del flujo de registro por invitación
Jobs programados	pg_cron (o node-cron en un worker dedicado si no hay pg_cron)	Cierre de semanas, generación de obligaciones, recordatorios
Almacenamiento de comprobantes	S3 compatible (AWS S3 / Cloudflare R2 / MinIO) con URLs pre- firmadas	No guardar binarios en la BD
Email	Resend (o SMTP)	Invitaciones,

Capa	Tecnología	Justificación
	genérico vía Nodemailer) con plantillas React Email	recordatorios, verificaciones
Gráficas	Recharts	Series temporales por semana
Tablas	TanStack Table v8	Filtros, orden, paginación server-side
Tests	Vitest (unitarios) + Playwright (E2E)	Cobertura mínima exigida en §16
Observabilidad	pino (logs JSON) + Sentry opcional	Auditoría y errores
Contenedores	Docker + docker- compose (app, postgres, minio, mailhog)	Entorno reproducible

2.1 Variables de entorno (.env.example)

```

DATABASE_URL=postgresql://user:pass@localhost:5432/refnet
AUTH_SECRET=<openssl rand -base64 32>
AUTH_URL=http://localhost:3000
APP_PUBLIC_URL=http://localhost:3000
S3_ENDPOINT=http://localhost:9000
S3_BUCKET=refnet-proofs
S3_ACCESS_KEY=
S3_SECRET_KEY=
S3_REGION=us-east-1
MAIL_FROM="Red <no-reply@example.com>"
RESEND_API_KEY=
TZ_DEFAULT=America/Caracas
ROOT_EMAIL=root@example.com
ROOT_PASSWORD=<solo para el seed inicial>
CRON_SECRET=<token para proteger /api/cron/*>

```

3. Modelo de Dominio

3.1 Conceptos

- **User:** persona registrada. Tiene un `sponsorId` (quien lo refirió). La raíz tiene `sponsorId = null`.

- **Árbol / Red:** grafo dirigido acíclico en forma de árbol estricto. Un usuario tiene **exactamente un** patrocinador.
- **Línea descendente (downline):** todos los descendientes de un usuario a cualquier profundidad.
- **Línea ascendente (upline):** cadena de patrocinadores hasta la raíz.
- **Nivel relativo:** profundidad del descendiente respecto a un ancestro (1 = hijo directo).
- **Invitation (link de referido):** token que permite registrarse bajo un patrocinador.
- **PaymentCycle (ciclo semanal):** ventana temporal (lunes 00:00 a domingo 23:59:59 en zona horaria configurable).
- **PaymentReport:** declaración de pago/donación hecha por un usuario para un ciclo.
- **Setting:** parámetro de negocio editable por el administrador.

3.2 Reglas de negocio (invariantes)

1. **R1** — Nadie puede registrarse sin un token de invitación válido, no expirado y no consumido (salvo la raíz vía seed).
2. **R2** — Un usuario no puede tener más hijos directos que `referral.max_direct_referrals` (valor efectivo = override individual del usuario, si existe; si no, el global). Esto se valida de forma atómica dentro de una transacción `SERIALIZABLE` y además con un *constraint* a nivel de BD (trigger) para evitar carreras.
3. **R3** — Las invitaciones **pendientes** cuentan contra el límite si `referral.count_pending_in_limit = true` (configurable, por defecto true). Esto evita sobre-emisión de cupos.
4. **R4** — Si `referral.max_depth > 0`, no se puede registrar a alguien cuya profundidad resultante exceda ese valor.
5. **R5** — El árbol es acíclico por construcción: un usuario nuevo siempre es hoja. Prohibido cambiar `sponsorId` salvo por una operación administrativa explícita (§10.5) que revalida ausencia de ciclos.
6. **R6** — Un usuario ve **únicamente** su propio subárbol (él y sus descendientes). La raíz ve todo. Ningún usuario ve a su upline ni a ramas laterales.
7. **R7** — Un usuario debe tener **a lo sumo** `payments.max_reports_per_cycle` reportes por ciclo (por defecto 1; configurable). Único índice parcial lo garantiza cuando el valor es 1.

8. **R8** — Los montos son `NUMERIC(14,2)`, nunca `float`. La moneda es configurable (`payments.currency`).
 9. **R9** — Un reporte en estado `APPROVED` o `REJECTED` no puede ser editado por su autor; solo revertido por un verificador (rol `ROOT/ADMIN`) dejando traza en `audit_log`.
 10. **R10** — Toda escritura relevante genera una entrada en `audit_log` (actor, acción, entidad, antes/después, IP, user-agent).
 11. **R11** — El cálculo de “semana” usa la zona horaria general `timezone` y el día de inicio `payments.week_start_day` (0=domingo ... 6=sábado; por defecto 1=lunes). **Nunca** usar la zona horaria del navegador para persistir.
 12. **R12** — Borrado de usuarios: **soft delete** (`deleted_at`). Nunca se rompe el árbol; un usuario desactivado conserva su posición y sus hijos.
-

4. Estrategia de Árbol (decisión técnica clave)

Se implementan **tres mecanismos complementarios** (redundantes a propósito, por rendimiento y por seguridad):

1. **Puntero al padre** (`users.sponsor_id`) -> verdad canónica.
2. **Materialized Path** (`users.path` tipo `ltree`, p. ej. `root.u3.u17.u92`) -> consultas de subárbol en $O(\log n)$ con índice GiST. Permite `path <@ 'root.u3'` para “todos los descendientes de `u3`”.
3. **Closure Table** (`user_closure(ancestor_id, descendant_id, depth)`) -> conteos por nivel, agregaciones y joins eficientes con pagos.

Para consultas puntuales también se admite **CTE recursiva**; se deja implementada como fallback y para verificación de consistencia.

4.1 Mantenimiento

- Al insertar un usuario: se calcula `path = parent.path || label`, `depth = parent.depth + 1`, y se insertan las filas de closure: (`a`, nuevo, `parent_depth+1`) para todo ancestro `a` de `parent`, más (nuevo, nuevo, 0).
- Se implementa con un **trigger AFTER INSERT** en `users` y una función SQL `fn_user_tree_insert()`, de modo que sea imposible crear un usuario con árbol inconsistente desde cualquier capa.
- Para el cambio de patrocinador (reasignación), función `fn_user_move(user_id, new_sponsor_id)` que recalcula paths y closure

del subárbol completo en una sola transacción y valida que new_sponsor_id no pertenezca al subárbol de user_id (anti-ciclo).

4.2 Consultas de referencia (SQL)

-- Subárbol completo de :id (sin incluirlo), con nivel relativo

```
SELECT u.*, c.depth AS relative_level
FROM user_closure c
JOIN users u ON u.id = c.descendant_id
WHERE c.ancestor_id = :id AND c.depth > 0 AND u.deleted_at IS NULL
ORDER BY c.depth, u.created_at;
```

-- Conteo de red por nivel

```
SELECT c.depth AS level, COUNT(*) AS total
FROM user_closure c JOIN users u ON u.id = c.descendant_id
WHERE c.ancestor_id = :id AND c.depth > 0 AND u.deleted_at IS NULL
GROUP BY 1 ORDER BY 1;
```

-- Variante con ltree (equivalente, útil para árboles muy anchos)

```
SELECT * FROM users WHERE path <@ (SELECT path FROM users WHERE id = :id) AND id <> :id;
```

-- Hijos directos con cupos restantes

```
SELECT u.id, u.full_name,
       (SELECT COUNT(*) FROM users ch WHERE ch.sponsor_id = u.id AND
        ch.deleted_at IS NULL) AS children,
       fn_effective_max_referrals(u.id) AS max_allowed
FROM users u WHERE u.sponsor_id = :id AND u.deleted_at IS NULL;
```

-- CTE recursiva (fallback / verificación)

```
WITH RECURSIVE t AS (
  SELECT id, sponsor_id, 0 AS lvl FROM users WHERE id = :id
  UNION ALL
  SELECT u.id, u.sponsor_id, t.lvl + 1 FROM users u JOIN t ON
  u.sponsor_id = t.id
) SELECT * FROM t WHERE lvl > 0;
```

5. Esquema de Base de Datos

Migración inicial en SQL puro

(prisma/migrations/0001_init/migration.sql). Extensiones

requeridas: CREATE EXTENSION IF NOT EXISTS ltree; CREATE
EXTENSION IF NOT EXISTS pgcrypto; CREATE EXTENSION IF NOT
EXISTS citext;

5.1 users

Columna	Tipo	Notas
id	uuid PK DEFAULT gen_random_uuid()	
public_code	text UNIQUE NOT NULL	código corto legible (8 chars, base32 sin ambigüedad) usado en links
sponsor_id	uuid NULL REFERENCES users(id)	NULL solo para la raíz
path	ltree NOT NULL	índice GiST
depth	int NOT NULL DEFAULT 0	0 = raíz
role	user_role NOT NULL DEFAULT 'MEMBER'	enum: ROOT, ADMIN, MEMBER
email	citext UNIQUE NOT NULL	
email_verified_at	timestampz NULL	
password_hash	text NOT NULL	Argon2id
full_name	text NOT NULL	
national_id	text NULL	cédula/DNI, único si profile.require_national_ id
phone	text NULL	E.164
birth_date	date NULL	
country	text NULL	ISO-3166-1 alpha-2
state / city	text NULL	
address	text NULL	
avatar_key	text NULL	clave en S3
status	user_status NOT NULL DEFAULT 'ACTIVE'	PENDING, ACTIVE, SUSPENDED
max_referrals_override	int NULL	si no es NULL, pisa el global (R2)
timezone	text NULL	si NULL usa el global

Columna	Tipo	Notas
last_login_at	timestampz NULL	
created_at / updated_at	timestampz NOT NULL DEFAULT now()	
deleted_at	timestampz NULL	soft delete

Índices: (sponsor_id), GiST (path), (email), (public_code), (status), (depth).

5.2 user_closure

Campo	Tipo	Notas
ancestor_id	uuid FK users	
descendant_id	uuid FK users	
depth	int NOT NULL	0 = sí mismo

PRIMARY KEY (ancestor_id, descendant_id), índices (descendant_id, depth), (ancestor_id, depth).

5.3 invitations

Columna	Tipo	Notas
id	uuid PK	
inviter_id	uuid FK users NOT NULL	
token_hash	text UNIQUE NOT NULL	se guarda SHA-256 del token; el token plano solo viaja en el link
kind	invite_kind NOT NULL DEFAULT 'MULTI'	SINGLE (un uso) o MULTI (link permanente del usuario)
email_target	citext NULL	si la invitación fue dirigida a un email concreto
max_uses	int NOT NULL DEFAULT 1	para MULTI se usa el cupo restante del usuario
uses	int NOT NULL DEFAULT 0	
expires_at	timestampz NULL	según referral.invite_ttl_hours

Columna	Tipo	Notas
revoked_at	timestampz NULL	
created_at	timestampz	

5.4 payment_cycles

Campo	Tipo	Notas
id	uuid PK	
period_key	text UNIQUE NOT NULL	formato ISO 2026-W41
starts_at	timestampz NOT NULL	
ends_at	timestampz NOT NULL	exclusivo
due_at	timestampz NOT NULL	ends_at + payments.grace_hours
status	cycle_status NOT NULL DEFAULT 'OPEN'	OPEN, GRACE, CLOSED
created_at	timestampz	

Restricción: EXCLUDE USING gist (tstzrange(starts_at, ends_at) WITH &&) -
> sin solapamientos.

5.5 payment_reports

Columna	Tipo	Notas
id	uuid PK	
user_id	uuid FK users NOT NULL	autor del pago
cycle_id	uuid FK payment_cycles NOT NULL	ciclo al que aplica
amount	numeric(14,2) NOT NULL CHECK (amount > 0)	
currency	text NOT NULL	copiado del setting al momento de crear
method	payment_method NOT NULL	enum configurable vía tabla payment_methods (ver 5.6)
method_id	uuid FK payment_methods	referencia real; method queda como texto desnormalizado para histórico

Columna	Tipo	Notas
reference	text NULL	nro. de referencia / hash de transacción
paid_at	timestampz NOT NULL	fecha en que el usuario dice haber pagado
reported_at	timestampz NOT NULL DEFAULT now()	fecha del registro en el sistema
note	text NULL	
proof_key	text NULL	comprobante en S3
status	report_status NOT NULL DEFAULT 'PENDING'	PENDING, APPROVED, REJECTED
reviewed_by	uuid FK users NULL	
reviewed_at	timestampz NULL	
review_note	text NULL	
snapshot_depth	int NOT NULL	profundidad del autor al momento del reporte (para reportes históricos)
snapshot_path	ltree NOT NULL	path del autor al momento del reporte
created_at / updated_at	timestampz	

Índices: (cycle_id, user_id), (user_id, paid_at DESC), (status), (reported_at DESC), GiST (snapshot_path). Índice único parcial (cuando payments.max_reports_per_cycle = 1): CREATE UNIQUE INDEX uq_report_user_cycle ON payment_reports(user_id, cycle_id) WHERE status <> 'REJECTED';

5.6 payment_methods (configurable por el admin)

Campo	Tipo	Notas	Extra	Más
id	uuid	code	label	instruct
PK	text	text	ions	require
	UNIQUE		text	s_refere
			NULL	nace
				bool
				DEFAU
				LT true
				DEFAU
				LT false
				LT true

5.7 settings (parámetros de negocio)

Columna	Tipo	Notas
key	text PK	p. ej. referral.max_direct_referrals
value	jsonb NOT NULL	valor tipado
type	setting_type NOT NULL	INT, DECIMAL, BOOL, STRING, ENUM, JSON
group_name	text NOT NULL	para agrupar en la UI
label	text NOT NULL	etiqueta visible
description	text NULL	ayuda contextual
min / max	numeric NULL	validación
options	jsonb NULL	para ENUM
is_public	bool DEFAULT false	si puede leerse sin autenticación (p. ej. nombre de la marca)
editable_by	user_role NOT NULL DEFAULT 'ROOT'	
updated_by	uuid FK users NULL	
updated_at	timestampz	

5.8 custom_fields + custom_field_values (campos definidos por el propio usuario)

Requisito explícito: **el administrador puede crear datos configurables sin llamar al programador.**

custom_fields:

Campos	Nombre	Extensión	Modo
id	en	co	la
u	tit	de	be
ui	y	te	l
d	cu	xt	te
P	st		xt
			X
			on
			ed
			e
			n
			n
			n
			to
			xt
			m
			m
			va
			er
			bo

Ca m po	Ti po	N ot as	Ex tr a	M ás													
K	o m _e nt ity (US E R, P A Y M E N T_ R EP O R T)			T, TE X T A R E A, N U M B E R, D EC I M A L, D A TE ,D A TE TI M E, B O O L, SE LE CT	b N U LL	bo ol	bo ol	re gi st ra tio n bo ol	pr ofi le bo ol	da sh bo ar d bo ol	us er _r ol e	N U LL	er ic N U LL	er ic N U LL	lu e js on b N U LL	in t	ol

Campo	Tipo	Notas	Extra	Más
				,M
				U
				LT
				IS
				EL
				EC
				T,
				FI
				LE
				,E
				M
				AI
				L,
				P
				H
				O
				N
				E,
				U
				R
				L)

UNIQUE (entity, code).

custom_field_values:

Campo	Tipo	Notas	Extra	Más
id uuid PK	field_id	entity_id	value jsonb	created_at
	uuid FK	uuid NOT NULL	NOT NULL	updated_at

UNIQUE (field_id, entity_id), índice GIN sobre value.

El motor de formularios **se genera dinámicamente** desde custom_fields: el schema Zod se construye en runtime (buildZodSchema(fields)), igual que las columnas de las tablas y los filtros del dashboard. Añadir un campo nuevo NO requiere despliegue.

5.9 Tablas de soporte

- `audit_log(id, actor_id, action, entity, entity_id, before jsonb, after jsonb, ip, user_agent, created_at)`
 - `notifications(id, user_id, type, title, body, data jsonb, read_at, created_at)`
 - `sessions, verification_tokens, password_resets` (estándar Auth.js)
 - `files(id, key, owner_id, mime, size_bytes, checksum, created_at)`
 - `cycle_obligations(id, cycle_id, user_id, status ('PENDING', 'FULFILLED', 'MISSED', 'EXEMPT'), expected_amount numeric(14,2), fulfilled_report_id uuid NULL)` -> materializa “quién debe pagar esta semana”, generada por el job de \$9.
 - `rate_limits(key, window_start, count)` o Redis si está disponible.
-

6. Catálogo de Configuración (seed obligatorio)

Insertar estas filas en settings. **Todas** editables desde `/admin/settings`.

Grupo referral

key	type	default	descripción
<code>referral.max_direct_referrals</code>	INT (1..1000)	12	Cupo de referidos directos por persona. Cambiarlo a 10 debe surtir efecto inmediato.
<code>referral.max_depth</code>	INT (0..200)	0	0 = ilimitado
<code>referral.count_pending_in_limit</code>	BOOL	true	Las invitaciones sin consumir ocupan cupo
<code>referral.invite_ttl_hours</code>	INT	168	Vigencia del link de un uso
<code>referral.allow_multiple_use_link</code>	BOOL	true	Link permanente por usuario
<code>referral.auto_approve_registration</code>	BOOL	true	Si false, el nuevo usuario queda

key	type	default	descripción
			PENDING hasta aprobación del patrocinador o admin
referral.require_email_verification	BOOL	true	
referral.allow_sponsor_reassign	BOOL	false	Habilita \$10.5

Grupo payments

key	type	default	descripción
payments.frequency	ENUM (WEEKLY,BIWEEKLY,MONTHLY)	WEEKLY	Frecuencia del ciclo
payments.week_start_day	INT (0..6)	1 (lunes)	
payments.grace_hours	INT	48	Plazo extra tras cerrar la semana
payments.expected_amount	DECIMAL	10.00	Monto esperado por ciclo
payments.amount_mode	ENUM (FIXED,MINIMUM,FREE)	MINIMUM	FIXED exige el monto exacto; MINIMUM exige >=; FREE acepta cualquiera
payments.currency	STRING	USD	
payments.max_reports_per_cycle	INT	1	
payments.require_proof	BOOL	false	Comprobante obligatorio
payments.require_reference	BOOL	true	
payments.allow_backdated_days	INT	7	Cuántos días atrás puede estar

key	type	default	descripción
			paid_at
payments.auto_aprove	BOOL	false	Si true, los reportes entran APPROVED
payments.exempt_root	BOOL	false	La raíz también reporta o no
payments.reminder_days_before_due	JSON	[1]	Recordatorios
payments.missed_grace_strikes	INT	3	Nº de ciclos incumplidos antes de SUSPENDED
payments.auto_spend_on_strikes	BOOL	false	

Grupo general

general.app_name (STRING, "Mi Red") · general.timezone (STRING, America/Caracas) · general.locale (ENUM es,en, default es) ·
 general.primary_color (STRING, #4F46E5) · general.logo_key (STRING) ·
 general.support_email (STRING) · general.terms_url (STRING)

Grupo visibility

key	default	descripción
visibility.downline_dept_h_visible	0 (ilimitado)	Cuántos niveles hacia abajo ve un miembro
visibility.member_sees_downline_payments	true	Un miembro ve los pagos de su red
visibility.member_sees_downline_amounts	true	Si false, ve el hecho del pago pero no el monto
visibility.member_sees_contact_data	true	Email/teléfono de su downline
visibility.mask_sensitive_fields	true	Enmascara cédula/teléfono parcialmente

Grupo security

security.password_min_length (8) · security.require_2fa_for_admin (false) · security.session_ttl_hours (720) · security.login_rate_limit_per_15m (10) · security.register_rate_limit_per_hour_per_ip (5)

Implementación: módulo lib/settings.ts con getSetting<T>(key), caché en memoria de 60 s, invalidación por NOTIFY settings_changed (LISTEN/NOTIFY de Postgres) para que un cambio surta efecto en todas las instancias sin reiniciar. fn_effective_max_referrals(user_id) en SQL devuelve COALESCE(users.max_referrals_override, settings['referral.max_direct_referrals']).

7. Roles y Permisos

Capacidad	ROOT	ADMIN	MEMBER
Ver todo el árbol	Sí	Sí	No (solo su subárbol)
Ver todos los pagos	Sí	Sí	solo propios + subárbol (según visibility.*)
Aprobar/rechazar pagos	Sí	Sí	No
Editar settings	Sí	solo editable_by<=ADMIN	No
Crear/editar custom_fields	Sí	Sí	No
Crear/revocar invitaciones propias	Sí	Sí	Sí
Reasignar patrocinador	Sí	No	No
Suspender usuario	Sí	Sí	No
Exportar CSV/XLSX	Sí	Sí	solo su subárbol

7.1 Guard de autorización (obligatorio en cada endpoint)

```
// lib/authz.ts
export async function assertCanViewUser(actor: Session, targetId:
string) {
  if (actor.role === 'ROOT' || actor.role === 'ADMIN') return;
  if (actor.userId === targetId) return;
  const row = await db.$queryRaw`
    SELECT depth FROM user_closure
    WHERE ancestor_id = ${actor.userId} AND descendant_id = $
{targetId}`;
  const maxDepth = await
getSetting<number>('visibility.downline_depth_visible');
  if (!row.length) throw new Forbidden();
  if (maxDepth > 0 && row[0].depth > maxDepth) throw new Forbidden();
}
```

Regla dura: ninguna consulta de lectura de red o pagos se ejecuta sin pasar por un `scopeFilter(actor)` que inyecta `JOIN user_closure c ON c.descendant_id = x.user_id AND c.ancestor_id = :actor` cuando el actor no es ROOT/ADMIN. Prohibido filtrar en el cliente. Adicionalmente, si se usa Postgres con RLS: políticas `USING (EXISTS (SELECT 1 FROM user_closure ...))`.

8. Flujos Funcionales

8.1 Seed de la raíz

1. Comando `pnpm seed:root` crea el usuario ROOT con `ROOT_EMAIL/ROOT_PASSWORD`, `sponsor_id = NULL`, `depth = 0`, `path = 'root'`.
2. Inserta settings, payment_methods base (Transferencia, Pago móvil, Zelle, Efectivo, Cripto) y el ciclo semanal actual.
3. Fuerza cambio de contraseña en el primer login.

8.2 Generación de link de referido

- Cada usuario tiene su link permanente:
`{APP_PUBLIC_URL}/r/{public_code}` (si `referral.allow_multi_use_link`).
- Además puede emitir invitaciones de un uso: `POST /api/invitations -> {APP_PUBLIC_URL}/invite/{token}`.

- Antes de emitir: validar cupo disponible = `fn_effective_max_referrals(u)` - (hijos activos + invitaciones pendientes si R3).
- La UI muestra: cupos usados / totales, lista de invitaciones pendientes, botón copiar, QR (generado con `qrcode`), y botón “enviar por email”.

8.3 Registro vía invitación

1. GET `/r/{code}` o `/invite/{token}` -> resuelve patrocinador; si inválido/expirado/sin cupo -> página de error explícita.
2. Formulario de registro: campos base (nombre completo, email, contraseña, teléfono, documento, fecha de nacimiento, país/estado/ciudad, dirección) + **todos los custom_fields con entity='USER' y show_in_registration=true**, renderizados dinámicamente con su validación.
3. POST `/api/auth/register` dentro de una transacción `SERIALIZABLE`:
 - revalida token y cupo (`SELECT ... FOR UPDATE` sobre el patrocinador),
 - valida R4 (profundidad),
 - crea users (el trigger llena path, depth y user_closure),
 - guarda custom_field_values,
 - consume la invitación (`uses + 1`),
 - crea cycle_obligations para el ciclo abierto,
 - escribe audit_log,
 - envía email de verificación si `referral.require_email_verification`.
4. Si `referral.auto_approve_registration = false` -> status = PENDING y se notifica al patrocinador para aprobar.
5. Manejo de error de concurrencia: si dos personas usan el último cupo a la vez, una recibe 409 REFERRAL_LIMIT_REACHED.

8.4 Portal del miembro (/portal)

- **Inicio:** tarjetas con *cupos restantes*, *tamaño total de mi red*, *mi estado de pago de esta semana* (con cuenta atrás al `due_at`), *total aportado por mi red en el ciclo*.
- **Mi red:**
 - *Vista Árbol:* componente `<NetworkTree>` colapsable, carga perezosa por nivel (`GET /api/network/children?parentId=`), búsqueda por

- nombre/código, badge de nº de descendientes y de estado de pago del ciclo. Para redes grandes usar virtualización (react-window).
- *Vista Tabla*: todos los descendientes con columnas: nivel relativo, nombre, patrocinador, email/teléfono (según `visibility.*`), fecha de registro, nº de referidos propios, estado del ciclo actual, total aportado histórico. Filtros + exportación.
- *Detalle de persona* (/portal/network/[id]): ficha completa con datos base, campos personalizados, su link de red, su propio subárbol y su historial de pagos. Acceso bloqueado por `assertCanViewUser`.
- **Reportar pago**: formulario con monto, método, referencia, fecha de pago, nota, comprobante (upload a S3 por URL pre-firmada) + `custom_fields` de `PAYMENT_REPORT`. Validaciones según `payments.*`.
- **Mis pagos**: historial con estado, motivo de rechazo y botón “corregir y reenviar” cuando esté `REJECTED`.
- **Mi perfil**: edición de datos y campos personalizados.

8.5 Dashboard de la Persona #1 (/admin)

Requisito explícito del cliente: la raíz ve **siempre y en detalle** todos los reportes de pago: monto, fecha, quién y cuándo.

KPIs (tarjetas): total recaudado (periodo), nº de reportes, ticket promedio, **tasa de cumplimiento** = reportes aprobados / obligaciones del ciclo, nº de pendientes por revisar, nº de incumplidos, usuarios nuevos en el periodo, tamaño total de la red.

Gráficas: - Barras: monto aprobado por semana (últimas 12/26/52 semanas, configurable en la UI). - Líneas: nº de reportes vs. obligaciones por semana. - Barras horizontales: aporte por nivel del árbol. - Top 10 contribuyentes y Top 10 patrocinadores por red. - Mapa de calor de cumplimiento (semanas × niveles).

Tabla maestra de pagos (server-side, paginada, exportable a CSV/XLSX): Semana (period_key) · Fecha de pago · Fecha de reporte · Usuario (link) · Código · Nivel · Patrocinador · Monto · Moneda · Método · Referencia · Estado · Comprobante · Revisado por · Revisado el · Nota

Filtros obligatorios (combinables, persistidos en la URL como query params): 1. **Semana / ciclo** (selector de period_key, multi-selección) 2. Rango de fechas personalizado (sobre paid_at o reported_at, a elección) 3. Estado (`PENDING` / `APPROVED` / `REJECTED`) 4. Usuario (autocompletar por nombre, email o código) 5. **Subárbol**: “pagos de la red de X” (usa `snapshot_path <@ path(X)`) 6. Nivel exacto o rango de niveles 7. Método de pago 8. Rango de montos (min/max) 9. Moneda

10. Con/sin comprobante 11. Texto libre en referencia/nota 12. Cualquier `custom_field` marcado `show_in_dashboard`

Acciones en lote: aprobar, rechazar (con motivo), exportar selección, reasignar ciclo.

Vista “Cumplimiento semanal”: matriz usuario × semana con celdas verde (aprobado) / amarillo (pendiente) / rojo (incumplido) / gris (exento), con drill-down al reporte.

Otras secciones admin: Usuarios (buscar, ver ficha, suspender, override de cupo, reasignar patrocinador), Invitaciones, Métodos de pago, **Configuración** (\$6, agrupada por `group_name`, con validación por `min/max/options` y confirmación al guardar), **Campos personalizados** (CRUD con previsualización del formulario), Ciclos (abrir/cerrar manualmente, regenerar obligaciones), Auditoría.

9. Ciclos y Tareas Programadas

Job	Cuándo	Qué hace
cycle:ensure	cada hora	Garantiza que exista el ciclo que contiene <code>now()</code> ; crea los siguientes 4 por adelantado. Usa <code>payments.frequency</code> y <code>payments.week_start_day</code> .
cycle:obligations	al crear un ciclo	Inserta <code>cycle_obligations</code> para todos los usuarios <code>ACTIVE</code> (excluye la raíz si <code>payments.exempt_root</code>), con <code>expected_amount = payments.expected_amount</code> .
cycle:reminders	diario 09:00 (TZ global)	Notifica (email + in-app) a quien no haya reportado, según <code>payments.reminder_day</code>

Job	Cuándo	Qué hace
cycle:close	cada hora	s_before_due. Si now() > due_at: marca obligaciones sin reporte aprobado como MISSED, pasa el ciclo a CLOSED, acumula <i>strikes</i> , y suspende si payments.auto_suspend_on_strikes.
tree:verify	diario 03:00	Compara user_closure/path contra la recursión sobre sponsor_id; si hay divergencia, registra alerta y reconstruye.
files:gc	semanal	Borra comprobantes huérfanos en S3.

Endpoints protegidos: POST /api/cron/{job} con header x-cron-secret.
Idempotentes (usar *advisory locks*: pg_try_advisory_lock).

9.1 Cálculo del ciclo (pseudocódigo)

```
import { DateTime } from 'luxon';
function cycleBounds(ref: Date, tz: string, weekStartDay: number,
freq: Freq) {
  const d = DateTime.fromJSDate(ref, { zone: tz });
  if (freq === 'WEEKLY') {
    const delta = (d.weekday % 7) - weekStartDay;           // luxon:
1=lun..7=dom
    const start = d.minus({ days: (delta + 7) % 7 }).startOf('day');
    return { start, end: start.plus({ weeks: 1 }), key: `$
{start.weekYear}-W${String(start.weekNumber).padStart(2, '0')}` };
  }
  // BIWEEKLY: anclado a payments.epoch_date; MONTHLY:
startOf('month')
}
```

Nunca usar new Date() sin zona. Persistir siempre en UTC (timestampztz), formatear en la zona del usuario o la global.

10. API (contratos)

Todas las respuestas: { ok: true, data } o { ok: false, error: { code, message, details? } }. Validación de entrada con Zod en cada handler.

Paginación: ?page=&perPage=&sort=&dir=.

10.1 Auth

- POST /api/auth/register -> { token|code, email, password, fullName, phone?, nationalId?, birthDate?, country?, state?, city?, address?, custom: Record<code, any> }
- POST /api/auth/login, POST /api/auth/logout, POST /api/auth/forgot, POST /api/auth/reset, GET /api/auth/verify?token=
- GET /api/public/invite/:token -> { inviterName, inviterCode, slotsLeft, registrationFields[] } (sin autenticación, sin filtrar datos sensibles)

10.2 Invitaciones

- GET /api/invitations (propias) · POST /api/invitations { kind, emailTarget?, expiresInHours? } · POST /api/invitations/:id/revoke · POST /api/invitations/:id/resend

10.3 Red

- GET /api/network/summary -> { totalDownline, byLevel[], slotsUsed, slotsMax, directChildren }
- GET /api/network/children?parentId=&page= -> hijos directos (carga perezosa del árbol)
- GET /api/network/tree?rootId=&maxDepth= -> subárbol anidado (cap por visibility.downline_depth_visible)
- GET /api/network/members?level=&q=&status=&cycle=&page= -> tabla plana del subárbol
- GET /api/network/members/:id -> ficha + campos personalizados + resumen de pagos
- GET /api/network/export?format=csv|xlsx

10.4 Pagos

- POST /api/payments { amount, methodId, reference?, paidAt, note?, proofKey?, custom }
- PATCH /api/payments/:id (solo autor y solo si PENDING)

- GET /api/payments?scope=mine|downline|all&cycle=&from=&to=&status=&userId=&subtreeOf=&level=&methodId=&minAmount=&maxAmount=&hasProof=&q=&page=&sort=
- POST /api/payments/:id/review { decision: 'APPROVED' | 'REJECTED', note? } (ROOT/ADMIN)
- POST /api/payments/bulk-review { ids[], decision, note? }
- GET /api/payments/stats?groupBy=week|level|method|user&from=&to= -> series para las gráficas
- GET /api/payments/compliance?cycle= -> matriz de cumplimiento
- GET /api/payments/export?format=csv|xlsx&... (mismos filtros)
- POST /api/uploads/sign { filename, mime, size } -> URL pre-firmada (validar mime en image/*, application/pdf y tamaño máx. 10 MB)

10.5 Administración

- GET/PUT /api/admin/settings (PUT acepta lote { key: value }[], valida tipo/min/max/options, audita)
 - GET/POST/PATCH/DELETE /api/admin/custom-fields
 - GET/POST/PATCH /api/admin/payment-methods
 - GET /api/admin/users?... · POST /api/admin/users/:id/suspend|activate · PATCH /api/admin/users/:id (incluye maxReferralsOverride)
 - POST /api/admin/users/:id/move { newSponsorId } -> solo si referral.allow_sponsor_reassign; valida anti-ciclo y cupo del nuevo padre; reconstruye path/closure
 - GET/POST /api/admin/cycles · POST /api/admin/cycles/:id/close|reopen
 - GET /api/admin/audit?...
-

11. Estructura de Carpetas

/app

```
/(public)/page.tsx /r/[code]/page.tsx /invite/[token]/page.tsx
/(auth)/login /register /forgot /reset /verify
```

```
/(portal)/portal/{page,network/{page,[id]},payments/{page,new},profile,invitations}
```

```
/(admin)/admin/{page,payments,compliance,users/[id],invitations,cycles,
settings,custom-fields,payment-methods,audit}
/api/... (ver $10)
```

```
/components ui/ · network/{NetworkTree,NodeCard,TreeLazyLoader} ·
payments/{ReportForm,PaymentsTable,ReviewDialog}
      dynamic-form/{DynamicForm,FieldRenderer,buildZodSchema} ·
charts/ · filters/
/lib db.ts · settings.ts · authz.ts · cycles.ts · tree.ts · money.ts
· storage.ts · mailer.ts
      audit.ts · rate-limit.ts · export.ts · errors.ts · validators/
/prisma schema.prisma · migrations/ · seed.ts
/jobs index.ts + un archivo por job de $9
/tests unit/ · e2e/
/docker docker-compose.yml · Dockerfile
```

12. Especificación de UI/UX

- Idioma por defecto **español** (es-VE), con i18n vía next-intl y claves en messages/es.json / en.json.
 - Diseño responsive, mobile-first. Tema claro/oscuro. Color primario desde general.primary_color.
 - <NetworkTree>: nodos con avatar, nombre, código, nº de referidos, chip de estado del ciclo ((verde) pagó / (amarillo) pendiente / (rojo) incumplió), flecha de expansión que carga hijos on-demand, breadcrumb del path, botón “ver ficha”, y buscador que resalta y auto-expande la ruta hasta el resultado.
 - Los formularios de registro, perfil y reporte se arman desde custom_fields + campos base: un solo componente <DynamicForm fields={...}/>. Añadir un campo en el panel debe reflejarse al instante.
 - Todas las tablas: filtros en la URL, orden server-side, selección múltiple, densidad ajustable, selector de columnas visibles, botón exportar.
 - Estados vacíos y de error redactados en lenguaje claro (sin códigos técnicos).
 - Accesibilidad: foco visible, navegación por teclado en el árbol (flechas), aria-expanded, contraste AA.
 - Formato de moneda y fecha con Intl, usando payments.currency y general.timezone.
-

13. Seguridad

1. Contraseñas con **Argon2id** (memoryCost 19456, timeCost 2, parallelism 1).
 2. Tokens de invitación: 32 bytes aleatorios en base64url; se almacena solo sha256(token).
 3. Rate limiting en login, register, forgot, uploads/sign (según security.*).
 4. CSRF: cookies SameSite=Lax + verificación de origen en mutaciones.
 5. Cabeceras: CSP estricta, HSTS, X-Content-Type-Options, Referrer-Policy: strict-origin-when-cross-origin.
 6. Autorización **siempre en el servidor**; el cliente nunca decide alcance.
 7. Validación de subidas: mime real (magic bytes), tamaño, nombre saneado, bucket privado y URLs de descarga pre-firmadas con TTL corto.
 8. Enmascarado de datos sensibles (V-1234****) cuando visibility.mask_sensitive_fields.
 9. Sin PII en logs. audit_log guarda diffs, no contraseñas ni tokens.
 10. Montos exclusivamente con Decimal (decimal.js en el servidor, numeric en BD). Prohibido number para dinero.
 11. Prevención de enumeración: respuestas idénticas en “email no existe” vs “contraseña inválida”.
 12. **Nota legal**: el sistema es de *registro* de pagos/donaciones, no procesa dinero. Si en el futuro se integra una pasarela, aislar en un módulo aparte y revisar la normativa aplicable sobre esquemas de aportes en red.
-

14. Rendimiento

- Índices de \$5 son obligatorios.
- Dashboard: **vista materializada** mv_payment_weekly(cycle_id, level, method_id, status, total_amount, count) refrescada con REFRESH MATERIALIZED VIEW CONCURRENTLY cada 5 min + al aprobar un pago.
- GET /api/network/tree nunca devuelve el árbol completo sin paginar: tope de 500 nodos por llamada.
- Exportaciones > 10.000 filas: generación en background con descarga por enlace.
- Caché de settings 60 s con invalidación por LISTEN/NOTIFY.
- Objetivo: tabla de pagos con 1M de filas responde < 500 ms en p95.

15. Datos Semilla de Demostración

Script `pnpm seed:demo --breadth=12 --depth=3 --weeks=8`: - Crea la raíz, un árbol completo con el ancho y profundidad indicados ($12^3 = 1.872$ usuarios), - Genera 8 ciclos semanales pasados con ~85 % de cumplimiento y montos variables, - Mezcla estados PENDING/APPROVED/REJECTED y algunos comprobantes, - Imprime credenciales de acceso de la raíz y de 3 miembros de ejemplo en distintos niveles.

16. Criterios de Aceptación (QA)

#	Criterio
A1	Cambiar <code>referral.max_direct_referrals</code> de 12 a 10 impide inmediatamente el 11.º referido, sin redespliegue.
A2	Nadie puede registrarse sin invitación válida; token expirado/revocado/agotado devuelve error claro.
A3	Dos registros simultáneos sobre el último cupo: uno entra, el otro recibe 409.
A4	La raíz ve el 100 % de la red y de los pagos; un miembro de nivel 2 ve exclusivamente su subárbol (probado con petición directa a la API, no solo en la UI).
A5	El árbol se renderiza y expande correctamente a 5+ niveles con 1.000+ nodos sin bloquear el navegador.
A6	Un reporte semanal queda ligado al ciclo correcto según <code>week_start_day</code> y <code>timezone</code> , incluido el caso límite de domingo 23:59.

#	Criterio
A7	Con <code>max_reports_per_cycle = 1</code> el segundo reporte del mismo ciclo es rechazado por la BD, no solo por la UI.
A8	El dashboard filtra por semana y los totales coinciden con la suma de la tabla y con el CSV exportado.
A9	Crear un campo personalizado (tipo SELECT, obligatorio, visible en registro y dashboard) aparece en el formulario, se guarda, se puede filtrar y se exporta, sin tocar código.
A10	Cerrar un ciclo marca correctamente los incumplimientos y, si está activo, suspende tras N strikes.
A11	<code>tree:verify</code> no reporta divergencias tras 1.000 registros aleatorios.
A12	Toda mutación relevante aparece en <code>audit_log</code> con actor y diff.
A13	Reasignar patrocinador reconstruye <code>paths/closure</code> y es rechazada si crea un ciclo.
A14	Montos con decimales ($10.05 + 0.10$) suman exacto, sin error de punto flotante.

Cobertura mínima: 80 % en `lib/` (`tree`, `cycles`, `settings`, `authz`, `money`). E2E Playwright de los flujos: invitar -> registrar -> reportar -> aprobar -> ver en dashboard filtrado por semana.

17. Plan de Implementación por Fases

1. **F1 – Base:** repo, Docker, Prisma, migración inicial, extensiones, trigger de árbol, seed de root y settings, `lib/settings.ts`.
2. **F2 – Auth e invitaciones:** registro por link, verificación de email, validación de cupo con transacción serializable.

3. **F3 – Red:** closure/ltree, API de red, <NetworkTree>, tabla de miembros, ficha de persona, guard de autorización.
 4. **F4 – Pagos:** ciclos, obligaciones, formulario de reporte, subida de comprobantes, revisión.
 5. **F5 – Dashboard raíz:** KPIs, gráficas, tabla maestra con los 12 filtros, vista de cumplimiento, exportaciones.
 6. **F6 – Configurabilidad:** panel de settings, CRUD de custom_fields, motor de formularios dinámicos, métodos de pago.
 7. **F7 – Jobs y notificaciones:** cron, recordatorios, cierre de ciclo, strikes, tree:verify.
 8. **F8 – Endurecimiento:** seguridad, rendimiento, vista materializada, tests, seed de demo, documentación (README, docs/).
-

18. Instrucciones Finales para el Generador de Código

1. Entregar un **monorepo ejecutable** con `pnpm install && docker compose up -d && pnpm db:migrate && pnpm seed:root && pnpm dev`.
2. TypeScript strict: true, sin any implícito, sin @ts-ignore. 3Reglas de negocio centralizadas en lib/; los handlers de API solo orquestan.
3. Ningún número de negocio en el código: leerlo de settings. Si falta una clave, fallar de forma explícita con un mensaje claro.
4. Cada endpoint: validación Zod -> autenticación -> autorización -> transacción -> auditoría -> respuesta tipada.
5. Textos de interfaz en español, extraídos a messages/es.json.
6. Incluir README.md con arquitectura, diagrama ER, decisiones de diseño y guía de operación (cómo cambiar el límite de referidos, cómo crear un campo personalizado, cómo cerrar una semana manualmente).
7. Si algún requisito resulta ambiguo, implementar la opción **más configurable** y documentarla en docs/decisiones.md.